

برنامه کارآموزی آزمایشگاه رباتیک

شرح فعالیت‌های هفته سوم و چهارم: آشنایی با بازو، مدل‌سازی، TF و ابزارهای ROS 2

مقدمه

در دو هفته اول با محیط توسعه، مفاهیم پایه ROS 2، ساختار Nodeها و ارتباطات ROS آشنا شدید و سپس کار با TurtleBot3 و محیط شبیه‌سازی آن را تجربه کردید. در ادامه دوره، تمرکز اصلی از ربات متحرک به سمت بازوی رباتیک منتقل خواهد شد. پلتفرم مورد استفاده در آزمایشگاه، بازوی OpenMANIPULATOR-X است که در کنار TurtleBot3 Waffle قرار گرفته و در ادامه دوره برای انجام فعالیت‌های مربوط به manipulation و pick and place مورد استفاده قرار خواهد گرفت.

در کار با یک بازوی رباتیک، علاوه بر Nodeها و Topicها، مفاهیم دیگری مانند مدل ربات، مفصل‌ها، مختصات، TF، نمایش سه‌بعدی، launch file، تست نرم‌افزار و کنترل حرکت بازو اهمیت پیدا می‌کنند. هدف این دو هفته این است که ابتدا ساختار نرم‌افزاری و هندسی یک ربات را بهتر درک کنید و سپس بتوانید از این مفاهیم برای اجرای یک سیستم ساده manipulation استفاده کنید.

در پایان این هفته باید بتوانید:

- ساختار کلی یک بازوی رباتیک و اجزای اصلی آن را توضیح دهید.
- تفاوت بین لینک‌ها، مفصل‌ها و End Effector را درک کنید.
- مفهوم TF و ارتباط بین Frameهای مختلف یک ربات را درک کنید.
- Transform بین Frameهای مختلف را با ابزارهای ROS 2 بررسی کنید.
- ساختار کلی فایل‌های URDF را درک کنید و مدل یک ربات را در RViz2 مشاهده و بررسی کنید.
- ارتباط بین URDF، TF و نمایش ربات در RViz2 را توضیح دهید.
- یک package ROS 2 را با استفاده از تست‌های مناسب بررسی کنید.
- بازوی OpenMANIPULATOR-X را در محیط شبیه‌سازی اجرا و بررسی کنید.
- ساختار Nodeها، Topicها و Actionهای مربوط به بازو را بررسی کنید.
- یک عملیات ساده مربوط به حرکت بازو و End Effector را اجرا و بررسی کنید.

در ادامه فعالیت‌های هفته سوم و چهارم به ترتیب شرح داده می‌شود.

هفته سوم: آشنایی با ساختار نرم‌افزاری و هندسی ربات

Launch

در هفته‌های قبل برای اجرای TurtleBot3 چندین Node و package را اجرا کردید. در یک سیستم واقعی، اجرای تعداد زیادی Node با وارد کردن تک‌تک دستورات کار مناسبی نیست. در ROS 2 می‌توان چند Node و تنظیمات مورد نیاز یک سیستم را در قالب یک Launch File تعریف کرد و سپس کل سیستم را با یک دستور اجرا کرد.

بخش مربوط به Launch را از مستندات رسمی ROS 2 مطالعه کنید:

[ROS 2 Documentation — Launch](#)

در این بخش ابتدا نمونه‌های موجود در مستندات را اجرا کنید و سپس بررسی کنید که Launch File دقیقاً چه کاری انجام می‌دهد.

تمرکز شما باید روی مفاهیم زیر باشد:

- اجرای چند Node با یک Launch File
- تعریف argument
- تغییر پارامترهای Node ها
- Remapping
- استفاده از Launch File های موجود در package ها
- تفاوت اجرای مستقیم یک Node با اجرای آن از طریق Launch

در ادامه، یکی از Launch File های مربوط به TurtleBot3 را پیدا کرده و بررسی کنید که چه Node هایی توسط آن اجرا می‌شوند. هدف این بخش صرفاً یادگیری syntax فایل‌های Launch نیست. باید بتوانید هنگام مشاهده یک Launch File موجود، تشخیص دهید که چه سیستمی قرار است اجرا شود و هر بخش چه نقشی دارد.

TF2

در یک ربات، هر بخش ممکن است مختصات محلی خود را داشته باشد. برای مثال، در یک بازوی رباتیک می‌توان Frame‌هایی مانند base، لینک‌های مختلف بازو و End Effector را در نظر گرفت. برای کار با چنین سیستمی باید بتوانیم رابطه مکانی بین این Frame‌ها را مشخص و محاسبه کنیم. ROS 2 برای این کار از TF2 استفاده می‌کند.

بخش مربوط به TF2 را از مستندات رسمی ROS 2 مطالعه کنید:

[ROS 2 Documentation — TF2](#)

ابتدا مثال‌های ساده مستندات را اجرا کنید و سپس ساختار TF در یک سیستم رباتیک را بررسی کنید. در این بخش باید مفاهیم زیر را درک کنید:

- Frame چیست؟
- Transform چیست؟
- ارتباط بین Parent Frame و Child Frame
- Rotation و Translation
- TF Tree
- تفاوت TF و Topic‌های معمولی
- Static Transform
- Dynamic Transform

پس از مطالعه، TF Tree مربوط به TurtleBot3 شبیه‌سازی‌شده را بررسی کنید. برای مثال بررسی کنید:

- Frame مربوط به base ربات کدام است؟
- سنسور LiDAR نسبت به ربات در چه Frame‌ای قرار دارد؟
- چه Frame‌هایی به صورت مستقیم یا غیرمستقیم به base متصل هستند؟
- Transform بین دو Frame چگونه قابل مشاهده است؟

در ادامه همین بررسی را برای بازوی OpenMANIPULATOR-X انجام خواهید داد. هدف این بخش این نیست که صرفاً چند دستور مربوط به tf2 را حفظ کنید. باید بتوانید با مشاهده یک سیستم ROS، رابطه مکانی بین بخش‌های مختلف ربات را پیدا و تفسیر کنید.

URDF و مدل سازی ربات

بخش ی اینکه ROS بتواند ساختار یک ربات را بشناسد، باید اطلاعات مربوط به لینک ها، مفصل ها و ارتباط بین آن ها در اختیار سیستم قرار گیرد.

یکی از قالب های متداول برای توصیف ساختار ربات، URDF (Unified Robot Description Format) است.

بخش مربوط به URDF را از مستندات ROS 2 مطالعه کنید:

[ROS 2 Documentation — URDF](#)

در این بخش ابتدا نمونه ساده موجود در مستندات را بررسی کنید. تمرکز شما باید روی مفاهیم زیر باشد:

- Link
- Joint
- Child و Parent
- Fixed Joint
- Revolute Joint
- Joint limits
- Visual
- Collision
- Inertial
- ارتباط URDF با TF

سپس فایل URDF مربوط به ربات و افل مورد استفاده در آزمایشگاه را پیدا کنید و ساختار آن را بررسی نمایید. برای مثال مشخص کنید:

- ربات از چه Link هایی تشکیل شده است؟
- چه Joint هایی بین Link ها وجود دارد؟
- Joint های حرکتی چه نوعی هستند؟
- Base ربات کدام Link است؟
- End Effector چگونه در مدل تعریف شده است؟
- محدوده حرکت Joint های بازو چگونه مشخص شده است؟

در این مرحله لازم نیست بتوانید یک URDF کامل و پیچیده از ابتدا ایجاد کنید. هدف این است که بتوانید یک فایل URDF موجود را بخوانید و ساختار هندسی و مکانیکی ربات را از روی آن متوجه شوید.

RViz2

در هفته دوم با RViz2 در ارتباط با TurtleBot3 و داده‌های سنسورها مواجه شدید. در این مرحله از RViz2 برای بررسی مدل سه‌بعدی و ساختار هندسی ربات استفاده خواهید کرد.

بخش مربوط به RViz2 را از مستندات ROS 2 مطالعه کنید:

[ROS 2 Documentation — RViz2](#)

یک سیستم دارای URDF را در RViz2 اجرا کنید و مدل ربات را مشاهده نمایید. سپس بررسی کنید که تغییر وضعیت Joint‌های ربات چگونه در RViz2 نمایش داده می‌شود. در این مرحله باید بتوانید:

- مدل ربات را در RViz2 نمایش دهید.
- Frame‌های مختلف را مشاهده کنید.
- TF Tree را با مدل ربات مرتبط کنید.
- وضعیت Joint‌های ربات را بررسی کنید.
- تفاوت بین مدل هندسی ربات و ربات واقعی را درک کنید.

در ادامه تلاش کنید مدل OpenMANIPULATOR-X را در RViz2 اجرا کرده و بخش‌های مختلف آن را بررسی کنید. هدف این است که بتوانید ارتباط بین سه مفهوم زیر را به‌صورت عملی مشاهده کنید:

URDF → TF → RViz2

یعنی بتوانید توضیح دهید که مدل ربات چگونه تعریف شده، TF چگونه موقعیت بخش‌های مختلف آن را مشخص می‌کند و RViz2 چگونه این اطلاعات را برای نمایش ربات استفاده می‌کند.

ROS 2 در Testing

تا این مرحله بیشتر با اجرای package‌ها و بررسی رفتار آن‌ها کار کرده‌اید. در توسعه نرم‌افزار رباتیک، علاوه بر نوشتن کد، باید بتوانیم از درست کار کردن بخش‌های مختلف آن نیز اطمینان حاصل کنیم.

بخش مربوط به Testing را از مستندات ROS 2 مطالعه کنید:

[ROS 2 Documentation — Testing](#)

در این بخش با مفهوم تست نرم‌افزار در ROS 2 آشنا شوید و نمونه‌های ارائه‌شده در مستندات را اجرا کنید.

سپس یکی از package‌های ساده‌ای که در هفته‌های قبل ایجاد کرده‌اید را انتخاب کنید و حداقل یک تست برای بخشی از کد آن بنویسید.

هدف این بخش آشنایی عمیق با framework‌های تست نیست. هدف این است که از همین مرحله توسعه نرم‌افزار را به‌عنوان فرآیندی ببینید که در آن کد، build و test در کنار یکدیگر قرار دارند.

هفته چهارم: آشنایی با بازو و Manipulation

آشنایی با OpenMANIPULATOR-X

در هفته چهارم تمرکز اصلی روی بازوی رباتیک آزمایشگاه خواهد بود. OpenMANIPULATOR-X یک بازوی رباتیک چند درجه آزادی است که برای توسعه و آزمایش الگوریتم‌های manipulation استفاده خواهد شد. ابتدا مستندات رسمی OpenMANIPULATOR-X را مطالعه کنید و با ساختار سخت‌افزاری و نرم‌افزاری آن آشنا شوید. (نیاز به مطالعه تمامی بخش‌ها نیست)

[OpenMANIPULATOR-X — ROBOTIS Documentation](#)

در این مرحله موارد زیر را بررسی کنید:

- [Degree of Freedom](#) تعداد
- Jointهای بازو
- Linkهای مختلف
- End Effector
- موتورهای مورد استفاده
- نحوه کنترل Jointها
- ارتباط بازو با ROS

Manipulation

پس از آشنایی با ساختار بازو، وارد بخش Manipulation خواهید شد. مستندات مربوط به Manipulation در TurtleBot3 را مطالعه کنید:

[TurtleBot3 — Manipulation](#)

هدف این بخش آشنایی عملی با فرآیند استفاده از یک بازوی رباتیک برای انجام یک وظیفه است. ابتدا مثال‌های موجود در مستندات را در محیط شبیه‌سازی اجرا کنید و سپس ساختار نرم‌افزاری آن‌ها را بررسی نمایید. در هنگام اجرای مثال‌ها به موارد زیر توجه کنید:

- فرمان حرکت بازو از کجا تولید می‌شود؟
- چه Node ای مسئول کنترل Jointها است؟
- وضعیت فعلی Joint چگونه دریافت می‌شود؟
- End Effector چگونه کنترل می‌شود؟

- ارتباط بین موقعیت Cartesian و Joint های بازو چیست؟
- TF چه نقشی در این فرآیند دارد؟
- RViz2 چه اطلاعاتی از وضعیت بازو نمایش می دهد؟

در ادامه تلاش کنید یک حرکت ساده برای بازو تعریف و اجرا کنید. برای مثال می توانید بازو را بین چند وضعیت مشخص حرکت دهید و در هر وضعیت موارد زیر را بررسی کنید:

- زاویه Jointها
- موقعیت End Effector
- TF مربوط به End Effector
- وضعیت بازو در RViz2

در این مرحله هنوز هدف پیاده سازی یک الگوریتم پیچیده برای grasping یا motion planning نیست. هدف این است که زنجیره کامل فرمان تا حرکت بازو را درک کنید.

بررسی ارتباط Joint Space و Cartesian Space

برای کار با بازو لازم است تفاوت بین دو نوع نمایش حرکت را درک کنید. در Joint Space وضعیت بازو با مقادیر Jointها مشخص می شود؛ برای مثال زاویه هر Joint. در Cartesian Space موقعیت و جهت End Effector نسبت به یک Frame مرجع مورد نظر قرار می گیرد. با استفاده از بازوی شبیه سازی شده بررسی کنید که تغییر در زاویه Jointها چگونه باعث تغییر موقعیت End Effector می شود. سپس با استفاده از TF موقعیت End Effector را نسبت به Frame پایه بررسی کنید. در این مرحله لازم نیست الگوریتم های Forward Kinematics و Inverse Kinematics را به صورت کامل پیاده سازی کنید، اما باید مفهوم آنها را درک کنید:

Forward Kinematics: از وضعیت Jointها به موقعیت End Effector

Inverse Kinematics: از موقعیت مورد نظر End Effector به وضعیت Jointها

این مفاهیم در ادامه دوره و هنگام کار روی manipulation و pick-and-place مورد استفاده قرار خواهند گرفت.

در پایان این دو هفته، انتظار می رود درک شما از ROS 2 از سطح اجرای Node و مشاهده Topic فراتر رفته باشد و بتوانید یک سیستم رباتیک را از نظر ساختار نرم افزاری، مدل هندسی، ارتباط Frame ها و جریان فرمان و داده تحلیل کنید.

